

Applied Exercise 1 — Lab Guide

Build a RAG Pipeline with Azure AI Foundry & Azure AI Search

AI Model Orchestration & RAG | Day 1

What You'll Build

In this lab, you'll stand up the full offline (indexing) half of a RAG pipeline in Azure, then run a ready-made Python app that queries it live: your question gets embedded, matched against your own documents in a vector index, and answered by a language model grounded in exactly what it found — with sources.

This mirrors the architecture from the "RAG Architecture: Ingestion Pipeline" and "Retrieval Pipeline" slides earlier today: you are building the offline half (documents → chunks → embeddings → index) by hand in the Azure portal, then running code that performs the online half (query → retrieve → augment → generate).

Time required

30–45 minutes.

What you'll end up with

- An Azure AI Foundry project with a chat model and an embedding model deployed
- An Azure AI Search service with a searchable, vectorized index of policy documents
- A working Python app on your own machine that answers questions grounded in those documents

Prerequisites

- An Azure subscription with permission to create resources (Contributor role or higher)
- Python 3.10 or later installed — check with: `python --version`
- Git installed — check with: `git --version`
- A code editor (VS Code recommended)
- The sample policy documents (10 .docx files) — provided by your instructor

NOTE

Creating these Azure resources will incur a small cost (typically a few dollars for the duration of the workshop). Please remove all the resources after practical.

Part A — Provision Your Azure Resources

STEP 1 Create a Foundry Resource

1. Go to <https://ai.azure.com> and sign in with your Azure account.
2. Select Create new, then choose New project (this provisions a Foundry resource for you if you don't already have one).
3. Give your project a name, for example rag-lab-<yourname>.
4. Choose a region that supports the models you'll deploy in Step 2 — East US, Sweden Central, and West Europe are safe defaults.
5. Select Create, and wait for the project to finish provisioning (usually under a minute).
6. On the project Overview page, copy the endpoint shown there — you'll need this as `AZURE_AI_FOUNDRY_ENDPOINT`.
7. Go to Keys and Endpoint (or the key icon on the Overview page) and copy a key — this is `AZURE_AI_FOUNDRY_API_KEY`.

TIP

The Foundry portal changes its left-hand navigation labels fairly often. If you can't find something, look for "...More" at the bottom of the left pane — it's usually hiding there.

STEP 2 Deploy AI Models

You need two model deployments: one chat model and one embedding model.

Deploy the chat model

1. In your Foundry project, go to Model catalog (or Models + endpoints).
2. Search for and select gpt-4o (or gpt-4o-mini for a cheaper/faster option).
3. Select Deploy, give the deployment a clear name (for example gpt-4o-chat), and confirm.

Deploy the embedding model

1. Repeat the same steps, this time searching for text-embedding-3-large (or text-embedding-3-small).
2. Name this deployment something clear too, for example text-embedding-3-large.

IMPORTANT

Write down the exact deployment names you chose, not the underlying model names. You'll enter these deployment names later as `AZURE_OPENAI_CHAT_DEPLOYMENT` and `AZURE_OPENAI_EMBEDDING_DEPLOYMENT` — they must match exactly, including capitalization.

STEP 3 Create an Azure AI Search Resource

1. Go to <https://portal.azure.com> and sign in.
2. Select Create a resource and search for Azure AI Search.

3. Select Create. Choose your subscription and resource group (create a new resource group for this lab if you don't have one).
4. Give the service a globally unique name.
5. Choose a pricing tier of Basic or higher — the Free tier does not support vector search at a usable scale for this lab.
6. Select Review + create, then Create, and wait for deployment to finish.
7. Once deployed, go to the resource's Overview page and copy the URL — this is `AZURE_SEARCH_ENDPOINT`.
8. Go to the Keys tab and copy the primary admin key — this is `AZURE_SEARCH_API_KEY`. (You'll only need an admin key for Step 6, when you build the index; a query key is enough once the index exists.)

STEP 4 Create a Storage Account

1. In the Azure portal, select Create a resource and search for Storage account.
2. Select Create. Use the same resource group as your Search resource.
3. Give it a globally unique name (lowercase letters and numbers only).
4. Leave Performance as Standard and Redundancy as Locally-redundant storage (LRS) — both are fine for a lab.
5. Select Review + create, then Create.

STEP 5 Upload the Policy Documents

1. Open your new storage account and select Containers from the left pane.
2. Select + Container, name it policy-documents, and set Public access level to Private.
3. Open the new container and select Upload.
4. Select all 10 Contoso HR policy .docx files and upload them.
5. Confirm all 10 files appear in the container before continuing.

STEP 6 Create an Index with Azure AI Search

This step uses Azure AI Search's built-in Import and vectorize data wizard, which handles chunking, calling your embedding model, and building the index for you — no code required.

1. Open your Azure AI Search resource in the Azure portal.
2. On the Overview page, select Import and vectorize data.
3. For the data source, choose Azure Blob Storage, then select your storage account and the policy-documents container.
4. For the vectorization source, choose your Foundry resource and select the embedding deployment you created in Step 2.
5. Leave the chunking settings at their defaults — they work well for this lab.
6. Give your index a name, for example hr-policy-index.
7. Select Create and wait for the indexer to finish running — this can take a few minutes for 10 documents.

- Once it completes, go to the Indexes tab, select your new index, and open the Fields tab.
- Note down the exact field names shown — particularly the key field, the content field, the title field, and the vector field.

IMPORTANT — FIELD NAMES VARY

Different versions of the wizard have named the vector field "vector" in some cases and "text_vector" in others. Your app's KEY_FIELD, CONTENT_FIELD, TITLE_FIELD, and VECTOR_FIELD constants (near the top of app.py) must match your index's actual field names exactly, or queries will fail or silently return nothing useful. Check the Fields tab — don't assume.

STEP 7 Collect Your Connection Details

Fill in this table as you go — you'll copy all seven values into a .env file in Part B.

Variable	Where to find it	Your value
AZURE_AI_FOUNDRY_ENDPOINT	Foundry project Overview page	
AZURE_AI_FOUNDRY_API_KEY	Foundry project Keys and Endpoint	
AZURE_OPENAI_EMBEDDING_DEPLOYMENT	The deployment name you chose in Step 2	
AZURE_OPENAI_CHAT_DEPLOYMENT	The deployment name you chose in Step 2	
AZURE_SEARCH_ENDPOINT	Azure AI Search resource Overview page	
AZURE_SEARCH_API_KEY	Azure AI Search resource Keys tab	
AZURE_SEARCH_INDEX	The index name you chose in Step 6	

Part B — Run the Demo App

STEP 8 Clone the Lab Repository

Open a terminal and run:

```
git clone <YOUR-GITHUB-REPO-URL>
cd <repo-folder-name>
```

Replace <YOUR-GITHUB-REPO-URL> with the repository URL your instructor shared.

STEP 9 Set Up a Python Environment

Create and activate a virtual environment, then install dependencies:

```
python -m venv venv
```

```
# Windows
venv\Scripts\activate

# macOS / Linux
source venv/bin/activate

pip install openai azure-search-documents python-dotenv
```

STEP 10 Configure Your .env File

1. Copy .env.example to a new file named exactly .env, in the same folder as app.py.
2. Open .env and fill in the seven values you collected in Step 7.

Your .env file should look like this (with your own real values, no quotes):

```
AZURE_AI_FOUNDRY_ENDPOINT=https://<your-resource>.services.ai.azure.com
AZURE_AI_FOUNDRY_API_KEY=<your-foundry-key>
AZURE_OPENAI_EMBEDDING_DEPLOYMENT=<your-embedding-deployment-name>
AZURE_OPENAI_CHAT_DEPLOYMENT=<your-chat-deployment-name>
AZURE_SEARCH_ENDPOINT=https://<your-service>.search.windows.net
AZURE_SEARCH_API_KEY=<your-search-key>
AZURE_SEARCH_INDEX=<your-index-name>
```

TIP

On Windows, File Explorer may hide the file extension and save this as ".env.txt" without telling you. If app.py can't find your variables, check the actual filename with `dir /a` in Command Prompt or `Get-ChildItem -Force` in PowerShell.

STEP 11 Run the App

```
python app.py
```

When prompted, type a question about Contoso India HR policy, for example:

- "How many weeks of paid paternity leave do employees get?"
- "What's the approval threshold for business expenses?"

You should see the retrieved chunks printed first, followed by a grounded answer and its sources. If you see that, your pipeline is working end to end.

Appendix — More Questions to Try

- What is Contoso India's policy on international remote work?
- How much does Contoso reimburse for home office equipment each year?

- What happens if I don't reconcile my corporate card statement on time?
- Who handles a sexual harassment complaint at Contoso India?
- What's the daily meal allowance for domestic business travel?
- Ask something the documents don't cover — confirm the app says it doesn't know, rather than guessing.